Group Project Code

Madeline, Maisie, Audrey

2026-05-11

1. Set up

Packages

```
# general use
library(tidyverse)
library(janitor)

# file organization
library(here)

# Boxplot Visualization
#install.packages("hrbrthemes")
library(hrbrthemes)
#install.packages("viridis")
library(viridis)

# Ridgeline Plot
#install.packages("ggridges")
library(ggridges)
#install.packages("ggplot2")
library(ggplot2)

# Color Palette
library(NatParksPalettes)
library(wesanderson)

# Use ragg_png device for better font rendering and system font support
knitr::opts_chunk$set(dev = "ragg_png")
```

Data

```
# weather data from NOAA weather station
weather <- read_csv(here("data", "NOAA-weather-data.csv"))

# water quality survey metadata
metadata <- read_csv(here("data", "NCOS_YSI_Water_Quality_Monitoring_0.csv"))

# water quality parameter measurements
water_parameters <- read_csv(here("data", "YSI_Data_Begin_1.csv"))
```

Code

```
weather_clean <- weather |>
  # made column names all lowercase
  clean_names() |>
  # puts date in proper format and makes sure its understood as date (not as chr)
  mutate(date = mdy(date)) |>
  # keeping only these 4 columns of interest
  select(date, prcp, tmax, tmin) |>
  # creating new columns for month and year
  # extracting month from the date (needs to be in the standard date format for this functio
  mutate(month = month(date),
         year = year(date)) |>
  mutate(wy = case_when(
    # when month os october or later, assign a value of year +1
    # in the water ywar for next year
    month >= 10 ~ year + 1,
    # keeping all months < 10 the same (sept or earlier), assigning value of existing year
    .default = year
    ))
```

```
metadata_clean <- metadata |>
  # col names easier to work with (all lowercase w/ _)
  clean_names() |>
  # "!" is the "not-operator", columns we DONT want to keep, excluding
  select(!c(object_id, creator, editor)) |>
  mutate(monitoring_date = mdy_hms(monitoring_date)) |>
  rename(monitoring_datetime = monitoring_date) |>
```

```
  mutate(month = month(monitoring_datetime),
         year = year(monitoring_datetime)) |>
  # assigning water year
  mutate(wy = case_when(
    month >= 10 ~ year +1,
    .default = year )) |>
  # creatig new column for site full name
  mutate(site_full = case_when(
    site_name == "east_channel" ~ "East Channel",
    site_name == "phelps_bridge" ~ "Phelps Bridge",
    site_name == "venoco_bridge" ~ "Venoco Bridge",
    site_name == "other" ~ "Other"
  )) |>
  # filter out Other observations
  filter(site_full != "Other")
```

```
# creating a new object from water parameters
water_parameters_clean <- water_parameters |>
  # cleaning column names
  clean_names() |>
  # "!" not operator, excluding unneeded columns
  select(!c(creator, editor)) |>
  # renaming depth code to be elevation code to better represent methodology
  rename(elevation_code = depth_code) |>
  ## Everything below came from visualizing the data first and defining what to fix/add
  # fixing a mistake in the data
  mutate(elevation_code = case_when(
    # anytime theres a 10 in elevation code, replace it with a 1
    elevation_code == 10 ~ 1,
    # everything else, keep the same
    .default = elevation_code)) |>
  mutate(
    # turning column into a factor
    elevation_code = as_factor(elevation_code),
    # defining the correct order for the levels of the factor
    elevation_code = fct_relevel(elevation_code,
                                 # write in the order you want
                                 "0", "1", "2", "3", "4", "5", "6"))
```

2. Joining

water quality = metadata + water parameters

```
# full joining
water_quality <- full_join(
  # left data frame
  x = metadata_clean,
  # right data frame
  y = water_parameters_clean,
  # name the columns in each df that you want to join by (naming the shared column)
  by = c("global_id" = "parent_global_id")) |>
  select(
    # information about the observation
    global_id, monitoring_datetime, time_of_wq_measurement, monitor_name, weather, site_name
    # water information
    water_surface_elevation, flow, notes,
    #measurement information
    elevation_code, do_mg_l, do_percent_sat, conductivity_u_s_cm, salinity_ppt, temperature_
  ) |>
  # taking the date out of this column, dont want the time
  mutate(date = date(monitoring_datetime)) |>
  # moving the new date column to a better location within the df
  relocate(date, .after = monitoring_datetime) |>
  # dont need this column anymore
  select(!monitoring_datetime)
```

water quality weather = water quality + weather

```
# left join
water_quality_weather <- left_join(
  # left data frame
  x = water_quality,
  # right data frame
  y = weather_clean,
  # joining these dfs by their date column
  by = "date")
```

3. Wrangling

Dealing with Outliers in Salinity

```
# creating new obj
salinity_outlier_ids <- water_quality_weather |>
  # filtering to only include outlier values
  filter(salinity_ppt > 1410) |>
  # extracting the global id column as a vector usign pull
  pull(global_id)
```

Get precipitation per month (site independent variable)

```
# create new object from `water_quality_weather`
monthly_prcp <- water_quality_weather |>
  # only include elevation codes 0, 1, and 2
  filter(elevation_code %in% c("0", "1", "2")) |>
  # get rid of site names w/ "other"
  filter(!site_name == "other") |>
  # removing any duplicate rows based on the combination of month, year, and wy
  distinct(month, year, wy, .keep_all = TRUE) |>
  # grouping by month and year
  group_by(month, year) |>
  # calculating the sum of prcp per month (not per site as it is a weather event and not site
  summarise(sum_prcp = sum(prcp, na.rm = TRUE)) |>
  # ungrouping
  ungroup() |>
  # creating a date column to be read as a date
  mutate(date = lubridate::make_date(.data$year, .data$month, 1)) |>
  # creating levels to `month`, putting them in a logical order
  mutate(month = factor(month,
                        levels = 1:12,
                        labels = c("Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "S
                        ordered = TRUE))
```

```
`summarise()` has grouped output by 'month'. You can override using the
`.groups` argument.
```

Filtering Salinity & Summarizing Variables (for final viz)

```
salinity_filtered <- water_quality_weather |>
  # keeping only elevation codes 0, 1, and 2
  filter(elevation_code %in% c("0", "1", "2")) |>
  # filtering the column global_id to exclude the values that match those in `salinity_outlie
  filter(!global_id %in% salinity_outlier_ids) |>
  # dropping the column for other_site_names
  select(!other_site_name) |>
  # get rid of site names w/ "other"
  filter(!site_name == "other") |>
  # setting the order of the levels (from lowest tp highest avg salinity value)
  # helps the order of the visualizations
  mutate(site_full = fct_relevel(site_full,
                                 "Phelps Bridge", "East Channel", "Venoco Bridge")) |>
  # taking out sites w/ NA
  filter(!site_full == "NA") |>
  # keeping only columns of interest (to reduce confusion)
  select(site_full, water_surface_elevation, elevation_code, salinity_ppt, temperature_c, mon
  # creating levels to `month`, putting them in a logical order
  mutate(month = factor(month,
                        levels = 1:12,
                        labels = c("Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "S
                        ordered = TRUE)) |>
  # group by site full, month, and year
  group_by(site_full, month, year) |>
  # calculate mean salinity (per site per month))
  summarize(mean_salinity = mean(salinity_ppt, na.rm = TRUE),
            .groups = "drop")
```

Joining Monthly Precipitation (sum) w/ Monthly Salinity (avg.) [for final viz]

```
salinity_prcp <- salinity_filtered |>
  left_join(monthly_prcp, by = c("month", "year"))
```

Making data frame for exploratory figures

```r
# create new object `exploratory_salinity` from `water_quality_weather`
exploratory_salinity <- water_quality_weather |>
  # only keep elevation codes 0, 1, 2, and 3 (keeping three just for visualization)
  filter(elevation_code %in% c("0", "1", "2", "3")) |>
  # filtering the column global_id to exclude the values that match those in `salinity_outlie
  filter(!global_id %in% salinity_outlier_ids) |>
  # dropping the column `other_site_names`
  select(!other_site_name) |>
  # filterign out site names that match "other"
  filter(!site_name == "other") |>
  # setting the order of the levels (from lowest tp highest avg salinity value)
  # helps the order of the visualizations
  mutate(site_full = fct_relevel(site_full,
                                 "Phelps Bridge", "East Channel", "Venoco Bridge")) |>
  # taking out sites w/ NA
  filter(!site_full == "NA") |>
  # keeping only columns of interest (to reduce confusion)
  select(site_full, water_surface_elevation, elevation_code, salinity_ppt, temperature_c, mon
  # group by site, elevation code, month, and year
  group_by(site_full, elevation_code, month, year) |>
  # calculate mean salinity
  summarize(mean_salinity = mean(salinity_ppt, na.rm = TRUE),
            .groups = "drop") |>
  # creating levels to `month`, putting them in a logical order
  mutate(month = factor(month,
                        levels = 1:12,
                        labels = c("Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "S
                        ordered = TRUE))
```

Water Surface Elevation

```r
# create a new object `monthly_surface_elv` from `water_quality_weather`
monthly_surfcae_elv <- water_quality_weather |>
  # only include elevation codes 0, 1, and 2
  filter(elevation_code %in% c("0", "1", "2")) |>
  # Filter out site names matchign to "other"
  filter(!site_name == "other") |>
  # group by site, month, and year
  group_by(site_full, month, year) |>
  # calculate mean water surface elevation
```

```
  summarise(avg_elv = mean(water_surface_elevation, na.rm = TRUE)) |>
  # ungroup data frame
  ungroup() |>
  # create a date column to be read as a date
  mutate(date = lubridate::make_date(.data$year, .data$month, 1)) |>
  # creating levels to `month`, putting them in a logical order
  mutate(month = factor(month,
                        levels = 1:12,
                        labels = c("Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug", "S
                        ordered = TRUE))
```

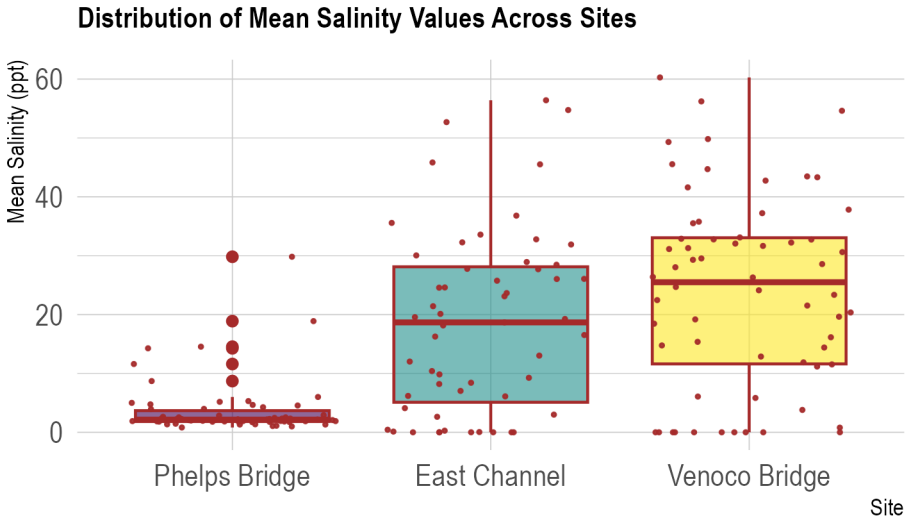
```
`summarise()` has grouped output by 'site_full', 'month'. You can override
using the `.groups` argument.
```

```
## Joining data frames
elv_viz <- monthly_surfcae_elv |>
  # joining by columns month, year, and site
  left_join(salinity_prcp,  by = c("month", "year", "site_full")) |>
  # filtering out all NA average elevation values
  filter(!avg_elv == "NaN")
```

4. Distribution Visualizations, Comparing Groups

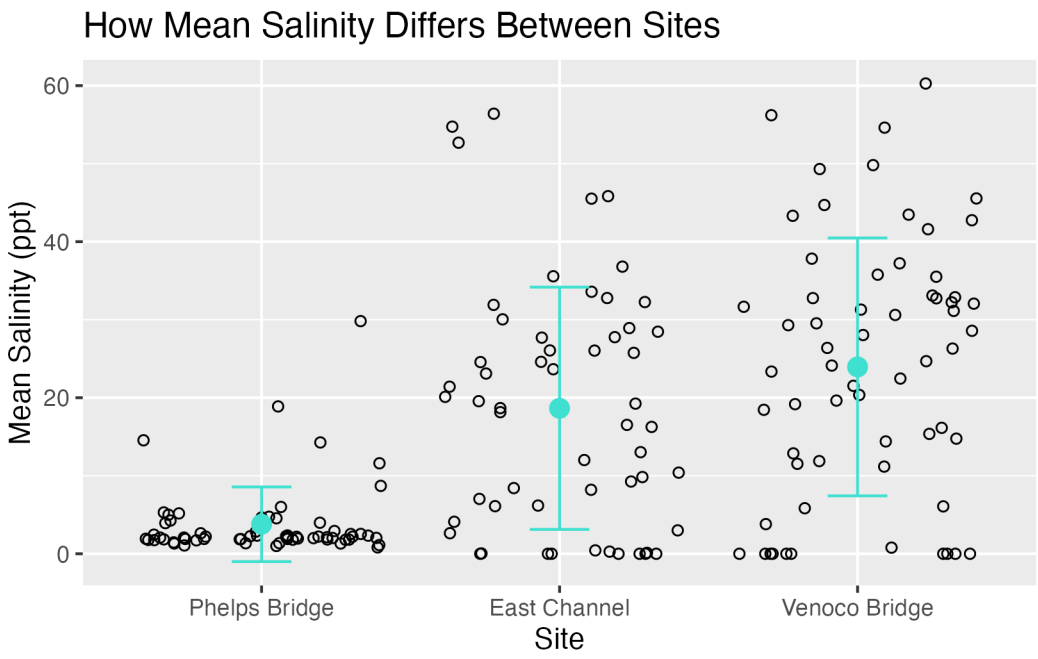
Boxplot w/ Jitter - Distribution of Mean Salinity Values Across Sites

```
# Plot
salinity_prcp %>%
  ggplot( aes(x= site_full, y=mean_salinity, fill = site_full)) +
    geom_boxplot(color = "brown") +
    scale_fill_viridis(discrete = TRUE, alpha=0.6) +
    geom_jitter(color="brown", size=0.4, alpha=0.9) +
    theme_ipsum() +
    theme(
      legend.position="none",
      plot.title = element_text(size=11)
    ) +
    ggtitle("Distribution of Mean Salinity Values Across Sites") +
    xlab("Site") +
  ylab("Mean Salinity (ppt)")
```



Strip Chart w/ Standard Deviation - How Mean Salinity Differs Between Sites

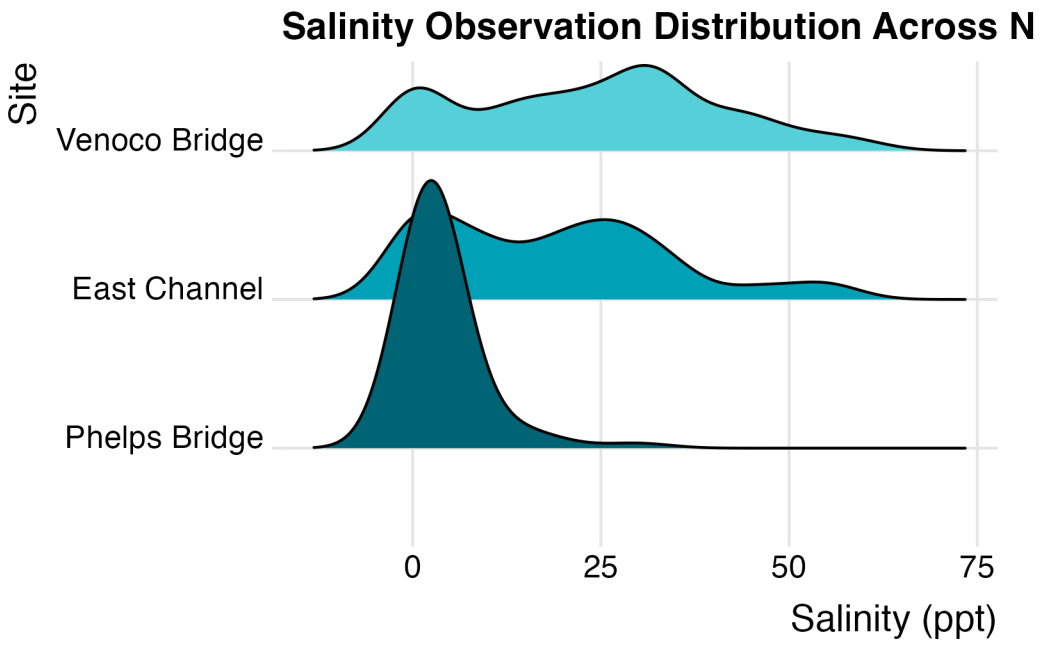
```
ggplot(data = salinity_prcp,
       mapping = aes(x = site_full,
                     y = mean_salinity)) +
  geom_jitter(shape = 21) +
  stat_summary(fun = mean, geom = "point", size = 3, color = "turquoise") +
  stat_summary(fun.data = mean_sdl, fun.args = list(mult = 1), geom = "errorbar", width = 0.1
  labs(x = "Site",
       y = "Mean Salinity (ppt)",
       title = "How Mean Salinity Differs Between Sites")
```



Ridgeline Plot - Salinity Observation Distribution Across NCOS Sites

```
ggplot(salinity_prcp, aes(x = mean_salinity, y = site_full, fill = site_full)) +
  geom_density_ridges() +
  theme_ridges() +
  theme(legend.position = "none") +
  scale_fill_manual(values = natparks.pals("Banff")) +
  labs(x = "Salinity (ppt)",
       y = "Site",
       title= "Salinity Observation Distribution Across NCOS Sites")
```

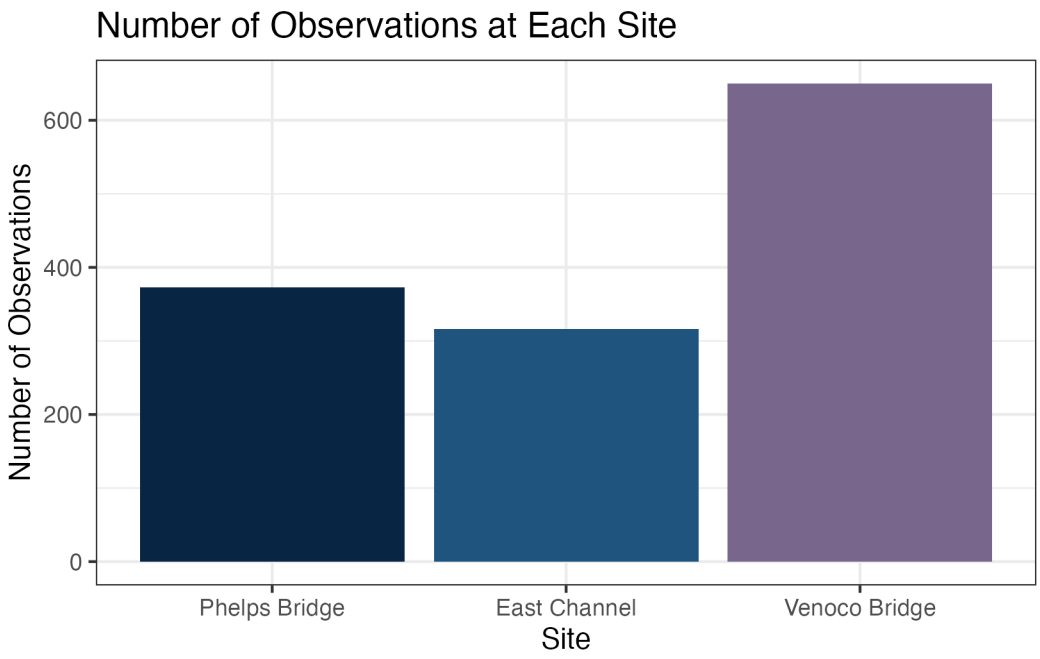
```
Picking joint bandwidth of 4.37
```



Bar Graphs (Distribution of Observations)

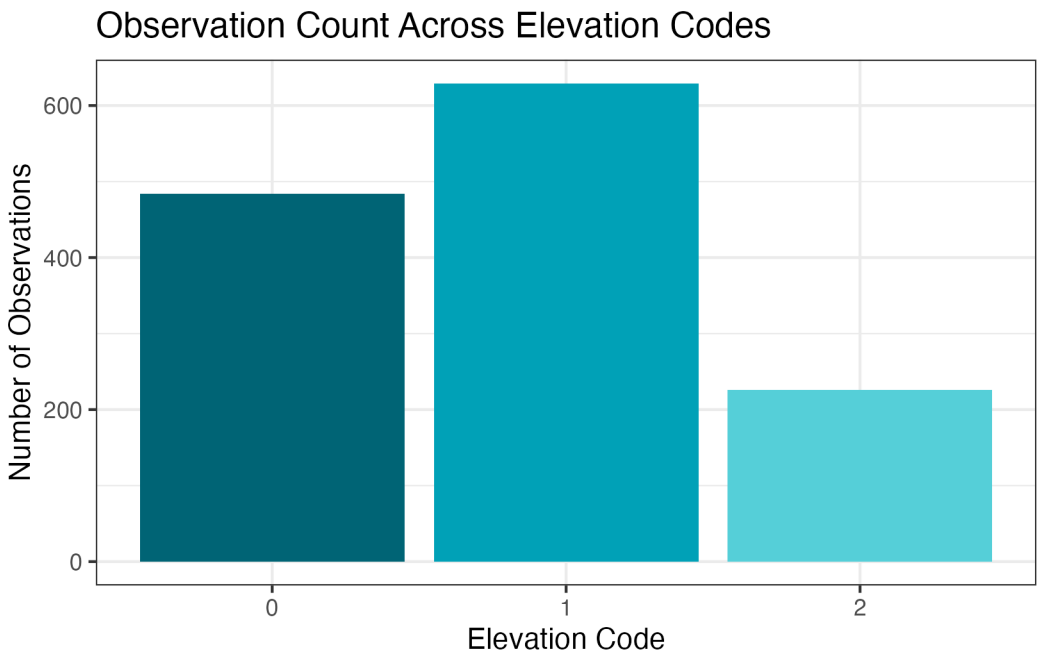
of Observations per Site

```
ggplot(data = water_quality_weather |>
         filter(!global_id %in% salinity_outlier_ids,
                site_name != "other",
                elevation_code %in% c("0", "1", "2")) |>
         mutate(site_full = fct_relevel(site_full,
                                        "Phelps Bridge", "East Channel", "Venoco Bridge")),
       mapping = aes(x = site_full)) +
  scale_fill_manual(values = natparks.pals("Volcanoes")) +
  geom_bar(aes(fill = site_full)) +
  theme_bw() +
  theme(legend.position = "none") +
  labs(x = "Site",
       y = "Number of Observations",
       title = "Number of Observations at Each Site")
```



# of Observations per Elevation Code

```
ggplot(data = water_quality_weather |>
         filter(!global_id %in% salinity_outlier_ids,
                site_name != "other",
                elevation_code %in% c("0", "1", "2")),
       mapping = aes(x = elevation_code)) +
  geom_bar(aes(fill = elevation_code)) +
  scale_fill_manual(values = natparks.pals("Banff")) +
  theme_bw() +
  theme(legend.position = "none") +
  labs(x = "Elevation Code",
       y = "Number of Observations",
       title = "Observation Count Across Elevation Codes")
```



 # Precipitation Peak

```
precipitation_peaks <- salinity_prcp |>
  group_by(year) |>
  slice_max(sum_prcp, n = 1, with_ties = FALSE) |>
  ungroup()
```

5. Figure Visualizing

Figure 1. - Total Monthly Precipitation at NCOS

```
ggplot(data = salinity_prcp,
       mapping = aes(x = date,
                     y = sum_prcp)) +
  geom_point(color = "darkblue") +
  geom_line(color = "darkblue") +
  theme_bw() +
  scale_x_date(date_breaks = "6 months", date_labels = "%b %Y") +
```

```
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  labs(x = "Date",
       y = "Total Monthly Precipitation",
       title = "Total Monthly Precipitation at NCOS") + geom_vline(data = precipitation_peaks
             aes(xintercept = date,
                 color = sum_prcp),
             linetype = "dotted",
             linewidth = 1)
```

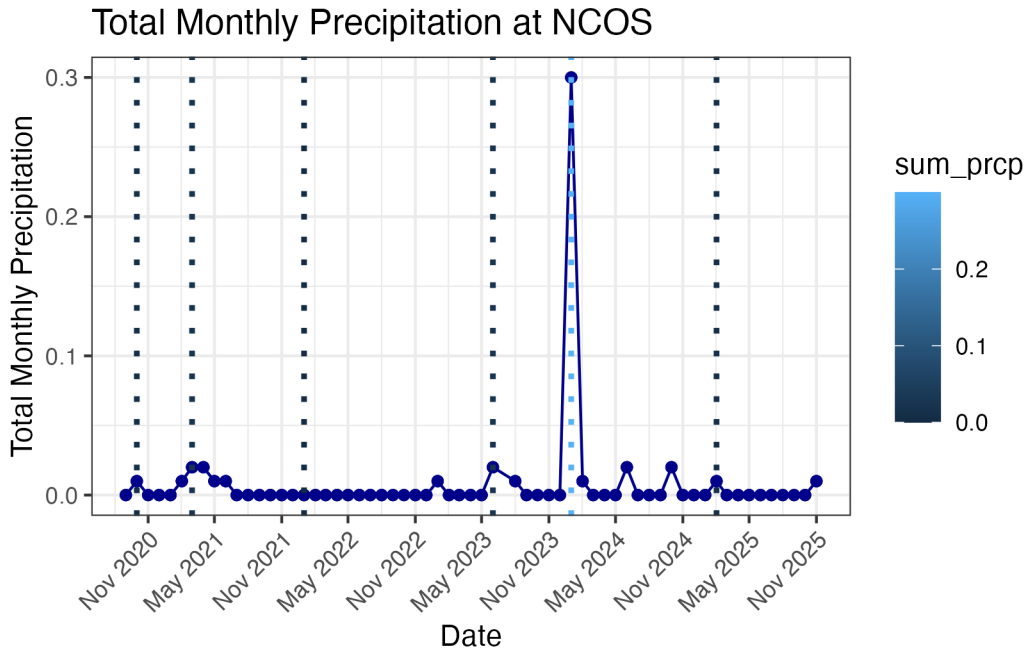


Figure 1b.

```
sitesalinity_directlabel <- ggplot(data = ,
                                   # x-axis: survey date
                                   # y-axis: total counts
                                   mapping = aes(x = observation_date,
                                                 y = total,
                                                 color = species)) +
  # basic geometries -----
  # points to represent counts at each survey
  geom_point() +
```

```
  # lines to connect points
  geom_line()
```

Figure 2. - How Salinity Relates to Water Surface Elevation (Venoco Bridge)

```
ggplot(data = elv_viz,
       mapping = aes(x = avg_elv,
                     y = mean_salinity)) +
  geom_point(color = "maroon") +
  facet_wrap(~ site_full) +
  scale_color_manual(values = wes_palette("GrandBudapest1")) +
  theme_bw() +
  labs(x = "Monthly Average Water Surface Elevation (m)",
       y = "Monthly Average Salinity (ppt)",
       title = "Figure 2. How Salinity Relates to Water Surface Elevation (Venoco Bridge")
```

Figure 2. How Salinity Relates to Water Surface Elevation (Venoco Bridge

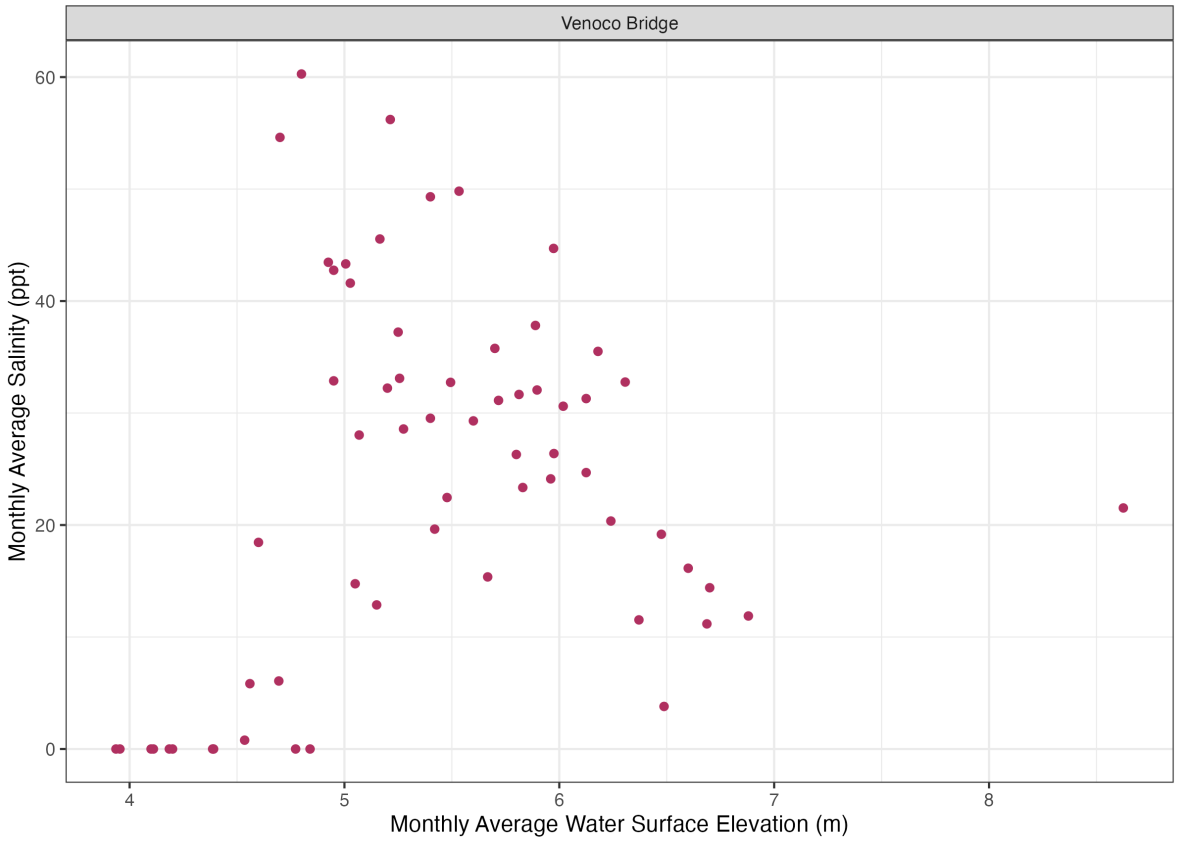


Figure 3. - How Salinity Varies Between Elevation Codes

```
ggplot(data = exploratory_salinity,
       mapping = aes(x = elevation_code,
                     y = mean_salinity)) +
  geom_point(shape = 21) +
  stat_summary(fun = median,
               geom = "point",
               color = "maroon") +
  stat_summary(fun = median,
               geom = "line",
               color = "maroon",
               group = 1) +
  labs(x = "Elevation Code",
       y = "Mean Salinity",
       title = "Figure 3. Mean Salinity Differences Between Elevation Codes") +
  facet_wrap(~ site_full) +
  theme_bw() +
  scale_x_discrete(breaks = c(0,1, 2, 3)) +
  scale_y_continuous(position = "right") +
  coord_flip()
```

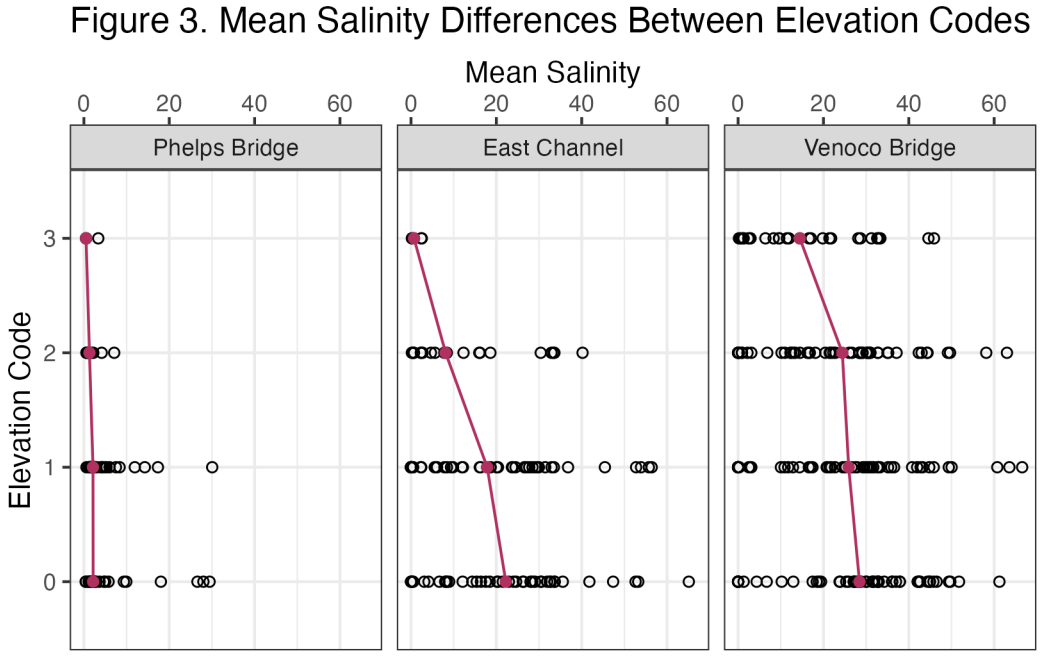


Figure 4. - How Precipitation Explains Salinity

```
ggplot(data = salinity_prcp,
       mapping = aes(x = sum_prcp,
                     y = mean_salinity,
                     color = site_full)) +
  geom_point() +
  facet_wrap(~ site_full) +
  scale_color_manual(values = wes_palette("Royal2")) +
  theme_bw() +
  labs(x = "Monthly Precipitation (Total)",
       y = "Monthly Average Salinity (ppt)",
       title = "Figure 4. How Precipitation Explains Salinity",
       color = "Site")
```